PAPER_177: FluidSolver Navier-Stokes + UQFF Coupling — Quasar Jet Dynamics

Abstract

This paper presents a UQFF analysis of astrophysical observables, deriving compressed field equations and observational predictions within the Star-Magic/UQFF framework.

Whitepaper §2.4-I | Thread 381a8fe7 | Session 48

Abstract

The CoAnQi codebase implements a 2D incompressible Navier-Stokes solver (FluidSolver) on a 32×32 grid, driven by the UQFF resonance gravity as an external body force. This enables simulation of quasar jet dynamics as a coupled UQFF-fluid system. This paper documents the solver algorithm, boundary conditions, UQFF coupling interface, and jet injection mechanism.

UQFF Discovery: Novel application of UQFF calibration constants (? = 5.0×10?4 day?¹, [SSq] = 0.57) uniquely enabling this analysis — establishing a new connection in the UQFF framework not present in Standard Model treatments.

$$F_U(r,t) = \sum_{i=1}^{4} U_{gi} + U_m + U_A - U_{b_i}, \quad \kappa = 5.0 \times 10^{-4}\,\text{day}^{-1},\ [SSq] = 0.57$$

$$U_{b_i}(r) = \kappa \cdot [SSq] \cdot \frac{GM}{r^2}, \quad \kappa = 5.0 \times 10^{-4}\,\text{day}^{-1},\ [SSq] = 0.57,\ \beta_i = 0.61$$

1. Configuration Parameters

```
class FluidSolver {
    const int N       = 32;       // Grid resolution (32×32)
    const double dt   = 0.1;      // Timestep [s]
    const double visc = 0.0001;   // Kinematic viscosity [m²/s]
    double force_jet  = 10.0;     // Jet injection force [N/m²]
    // State arrays: ux[N+2][N+2], uy[N+2][N+2], p[N+2][N+2], rho[N+2][N+2]
    // Previous: ux_prev, uy_prev, p_prev
};
```

---

2. Solver Algorithm

The solver follows the Stam (1999) stable fluids method with four phases:

Phase 1 — Add Source

```
ux[i][j] += dt × uqff_g   (external gravity from compute_resonance_MUGE)
```

Phase 2 — Diffuse

```
diffuse(N, 1, ux_prev, ux, visc, dt):
    a = dt × visc × N²
    for k in 0..19:  // 20 Gauss-Seidel iterations
        for i,j in 1..N:
            ux[i][j] = (ux_prev[i][j] + a*(ux[i-1][j]+ux[i+1][j]+ux[i][j-1]+ux[i][j+1]))
                       / (1 + 4a)
    set_bnd(N, 1, ux)
```

Phase 3 — Project (Pressure solve / Helmholtz decomposition)

```
project(N, ux, uy, p, div):
    h = 1.0 / N
    // Compute divergence
    div[i][j] = -0.5h*(ux[i+1][j]-ux[i-1][j] + uy[i][j+1]-uy[i][j-1])
    p[i][j] = 0
    // Pressure Poisson (20 iterations)
    for k in 0..19:
        p[i][j] = (div[i][j]+p[i-1][j]+p[i+1][j]+p[i][j-1]+p[i][j+1]) / 4
    // Subtract pressure gradient
    ux[i][j] -= 0.5*(p[i+1][j]-p[i-1][j]) / h
    uy[i][j] -= 0.5*(p[i][j+1]-p[i][j-1]) / h
    set_bnd(N, 1, ux); set_bnd(N, 2, uy)
```

Phase 4 — Advect (Semi-Lagrangian backtracing)

```
advect(N, b, d, d0, ux, uy, dt):
    dt0 = dt × N
    for i,j in 1..N:
        x = i - dt0*ux[i][j]    // backtrack position
        y = j - dt0*uy[i][j]
        // Clamp and bilinear interpolate d0 at (x,y)
        d[i][j] = bilinear_interp(d0, x, y)
    set_bnd(N, b, d)
```

---

3. Boundary Conditions (set_bnd)

```
b=1 (x-velocity): ux[0][j] = -ux[1][j]  (no-slip left/right walls)
b=2 (y-velocity): uy[i][0] = -uy[i][1]  (no-slip top/bottom walls)
b=0 (scalar):     zero-gradient Neumann on all walls
Corner cells: average of two adjacent boundary cells
```

---

4. UQFF Coupling Interface

```
void step(double uqff_g) {
    // uqff_g = compute_resonance_MUGE(muge_system)
    // Apply UQFF as external body force
    for i in 1..N:
        for j in 1..N:
            ux[i][j] += dt × uqff_g;    // gravity drives x-velocity
```

```
    std::swap(ux_prev, ux);
    diffuse(N, 1, ux_prev, ux, visc, dt);
    diffuse(N, 2, uy_prev, uy, visc, dt);
    project(N, ux, uy, p, div);
    std::swap(ux_prev, ux); std::swap(uy_prev, uy);
    advect(N, 1, ux, ux_prev, ux_prev, uy_prev, dt);
    advect(N, 2, uy, uy_prev, ux_prev, uy_prev, dt);
    project(N, ux, uy, p, div);
}
```

---

5. Jet Force Injection

```
void add_jet_force() {
    // Injects a transverse jet force at the domain midpoint
    for i in N/4..3N/4:            // Inject across central 50% of grid
        uy[i][N/2] += force_jet;  // force_jet = 10.0 N/m²
}
```

This models quasar jet ejection as a sustained transverse force at the equatorial plane, consistent with the UQFF interpretation of SCm expulsion igniting along the Ug4 galactic axis.

---

6. Velocity Field Visualisation

```
void print_velocity_field() {
    // ASCII: '#'=|v|>1, '+'=|v|>0.5, '.'=|v|>0.1, ' '=|v|=0.1
    for i in 1..N:
        for j in 1..N:
            mag = sqrt(ux[i][j]^2 + uy[i][j]^2)
            print( (mag>1)?'#': (mag>0.5)?'+': (mag>0.1)?'.':' ' )
}
```

---

7. integrate_fluids_for_muge() — System-Level Integration

```
void simulate_fluids_for_muge(MUGESystem& sys, int N_steps=100) {
    FluidSolver fs;
    for int step=0; step<N_steps; step++:
        double g = compute_resonance_MUGE(sys);  // UQFF gravity
        fs.add_jet_force();
        fs.step(g);
    fs.print_velocity_field();
}
```

This function is called from `populate_simulation_entities()` for each system in the 7-system canonical set, coupling fluid dynamics to UQFF.

---

8. Physical Interpretation

Solver Component	Physical Analogue
ux, uy velocity field	Quasar jet plasma velocity
uqff_g (resonance_MUGE)	UQFF gravitational drive
visc = 0.0001	Magnetised plasma viscosity
add_jet_force	SCm expulsion ignition event
project (pressure)	Magnetohydrodynamic pressure balance
set_bnd (no-slip)	Accretion disk boundary

---

9. References

- FluidSolver.h/cpp (thread 381a8fe7)
- Stam, J. (1999) “Stable Fluids” — SIGGRAPH proceedings
- PAPER_174 (resonance_MUGE provides uqff_g)
- PAPER_176 (SCm expulsion as jet trigger)
- PAPER_178 (3D simulation entities that host fluid simulations)

Key UQFF calibrated constants: $\kappa = 5.0\text{e-}4$ day^{-1}; [SSq] = 5.7e-1; H_SCm ≈ 9.9e-1; U_UA ≈ 1.0e-4; k_η = 1.0e-113; β_i ≈ 6.0e-1; G = 6.674e-11 N·m^2/kg^2